

Python Packages

Python 3

To use Python 3 on the cluster you can use a module that has it as the default. In BASH on the command line you will do

```
module load miniconda3
conda activate GEN220 # use an existing gen220 environment
```

Do this one time (make sure you did the step above to activate the environment)

```
python3 -m ipykernel install --user --name GEN220 --display-name "Python (GEN220)"
```

This will load an environment where python3 is the python installed. You can check that this works by doing `python -V` and see the version is 3.XX

You can setup your own conda environment

This will allow you to install python packages as well as any other pkgs you might want to.

Make your own.

```
mkdir ~/bigdata/.conda
ln -s ~/bigdata/.conda ~/.conda
conda create -y -n mygen220 python=3 mamba
source activate mygen220
conda config --add channels defaults
conda config --add channels bioconda
conda config --add channels conda-forge
mamba install biopython bcbio-gff

mamba install ipykernel
python3 -m ipykernel install --user --name gen220 --display-name "Python (GEN220)"
# Installed kernelspec gen220 in /rhome/USER/.local/share/jupyter/kernels/gen220
```

Python Libraries

Python has many built in packages already installed and a whole host of contributions that span everything from plotting data, interfacing with other datatypes.

- <https://docs.python.org/3/library/gzip.html>
- <https://docs.python.org/3/library/csv.html>
- PyBed tools (processing BED files) - <https://daler.github.io/pybedtools/>
- BioPython GFF parsing
- There is a nice list of available libraries <https://wiki.python.org/moin/UsefulModules>

Gzip

Can open a file that is compressed or write to a file already compressed. This can save space for large files or when you get data from a resource without having to decompress it.

```
import gzip
with gzip.open(file,"rt") as fh:
    for line in fh:
        print("The first line from uncompressed:", line)
        break
```

URL / Web requests directly

```
# this is a URL at the UniProt database to get a protein sequence based on
# accession number
import urllib.request
url="https://www.uniprot.org/uniprot/P10127.fasta"
seqdata = urllib.request.urlopen(url)
for line in seqdata:
    linestrip = line.decode('UTF-8').strip()
    print(linestrip)
```

BioPython - a library of modules for bioinformatics

[BioPython Tutorial](#)

Modules for Sequence data, BLAST parsing, Multiple alignments

Already installed on biocluster

To install on your own computer that you control, use conda (or the Python tool pip)

```
conda create -y -n gen220 # only need to do this once
source activate gen220
conda install biopython # if you forget to do the line above it will fail
```

Simple BioPython

```
import Bio
from Bio.Seq import Seq
my_seq = Seq("ATGAGTACACTAGGGTAA")

print(my_seq)

rc = my_seq.reverse_complement()
pep = my_seq.translate()
```

```
print("revcom is", rc)
print(pep)
```

Parsing sequence files

```
more /bigdata/gen220/shared/data/E3Q6S8.fasta
```

```
>tr|E3Q6S8|E3Q6S8_COLGM RNase P Rpr2/Rpp21/SNM1 subunit domain-containing protein OS=Collet
MAKPKSESLPNRHAYTRVSYLHQAAAYLATVQSPTSDSTTNSSQPGHAPHAVDHERCLET
NETVARRFVSDIRAVSLKAQIRPSPSLKQMMCKYCDLLVEGKTCSTTVENASKGGKKPW
ADVMVTKCKTCGNVKRFPVSAPRQKRRPFREQKAVEGQDTPAVSEMSTGAD
```

To process this file:

```
import sys
import Bio
from Bio import SeqIO
from Bio.Seq import Seq

# seqfile
filename = "/bigdata/gen220/shared/data/E3Q6S8.fasta"
for seq_record in SeqIO.parse( filename , "fasta"):
    print(seq_record.id)
    print(repr(seq_record.seq))
    print(seq_record.seq)
    print(len(seq_record))
```

This will output:

```
tr|E3Q6S8|E3Q6S8_COLGM
Seq('MAKPKSESLPNRHAYTRVSYLHQAAAYLATVQSPTSDSTTNSSQPGHAPHAVDH...GAD')
MAKPKSESLPNRHAYTRVSYLHQAAAYLATVQSPTSDSTTNSSQPGHAPHAVDHERCLETNETVARRFVSDIRAVSLKAQIRPSPSLKQMM
172
```

GenBank files: another sequence format

```
LOCUS      AJ240084                  1905 bp    DNA        linear    PRI 03-FEB-2000
DEFINITION Homo sapiens TRIM gene, promoter.
ACCESSION  AJ240084
VERSION    AJ240084.1  GI:6911579
KEYWORDS   T-cell receptor interacting molecule; TRIM gene.
SOURCE     Homo sapiens (human)
  ORGANISM Homo sapiens
            Eukaryota; Metazoa; Chordata; Craniata; Vertebrata; Euteleostomi;
            Mammalia; Eutheria; Euarchontoglires; Primates; Haplorrhini;
            Catarrhini; Hominidae; Homo.
REFERENCE  1
AUTHORS    Hubener,C., Mincheva,A., Lichter,P., Schraven,B. and Bruyns,E.
```

TITLE Genomic organization and chromosomal localization of the human gene
 encoding the T-cell receptor-interacting molecule (TRIM)
 JOURNAL Immunogenetics 51 (2), 154-158 (2000)
 PUBMED 10663578
 REFERENCE 2 (bases 1 to 1905)
 AUTHORS Huebener,C.
 TITLE Direct Submission
 JOURNAL Submitted (06-MAY-1999) Huebener C., Immunomodulation Laboratory,
 Institute for Immunology, University of Heidelberg, Im Neuenheimer
 Feld 305, Heidelberg, 69120, GERMANY
 FEATURES Location/Qualifiers
 source 1..1905
 /organism="Homo sapiens"
 /mol_type="genomic DNA"
 /db_xref="taxon:9606"
 /clone_lib="RPC11,3-5 Human PAC library"
 gene 1..1902
 /gene="TRIM"
 regulatory 1..1746
 /regulatory_class="promoter"
 /gene="TRIM"
 5'UTR 1747..1902
 /gene="TRIM"
 ORIGIN
 1 ccaaaaattt ccagtcctga aaccctttct ctttccaatg tcctctgtaa gctcgagttg
 61 tgggcatcta ctttgcccat attccaaggt cttgcttagg taacctctgt agtcctttct
 121 tgagcctagg acttctactt ttcttaccag ttacctctt tcaggaccaa agctcaactc
 181 ctcaaggcca taactaggcc ctctcctctc aaactgattt atcagggtgcc cgaatcttcc
 241 tgaatgtctg ggattcaact tttcagcagt cttcctccct acgttccatc taattctaag
 301 atgaaacctt ctgattcttt gttgtcctct gatccctaca tgaacctgag gctgctgttc
 361 cctgaagtct tgttctgtca gcatccaggc ctgcttcata aaacctgtca ctctgctaata
 421 ggttagcggc tgaacaaaga gtctctgtgc caaataagtt tagaaaaact ctgataaaaa
 481 tattatttgg gtttcctttt cgcaggactt acctaagcct ttaatatgca tctacggagg
 541 taaaaataaa gctatatatt ttttccaaag atatttggtg aagaaacatt tgtcttctgc
 601 gtttcttaaa ggccgagtgt tctatggaac atactttaaa aaaccctttt aaagaagctt
 661 agaccagaga atctccaagg tctctttcag ttttacagcc tctgagtcaa cgattcacca
 721 aaaaatattt tggggggaag tgattgaagt ggaaaaattt gttagtgttt agccagcttt
 781 gtccaaagga taagatgcac tgtattttgc ttactaggga gttattttct ataatggaag
 841 acaaagaaag cacaagacac ccatggtttt gtttggtcaa tctactgagag taagtctcaa
 901 ttattgagac ttacgatgtg cgggtgtgct taattctagt tatgaaattt taataatgaa
 961 taatatagat tctattcctt atatgagttt caaaagcat tgtccagaac atctatatta
 1021 aaatatctta tcatatacaa tatatgtaat ttaaaatgca ctcagaaaaat ctgcttggtta
 1081 aaatgcagat tctagtgtt caccctaaat agtctaattt agacggggcc aggatttta
 1141 actagcatct tatagcatat ttatgtacac caacatgtaa gaactgctgc tattaagatt
 1201 ctgggatggt gggttgagaac aggagcttgt tgtcagggtg ctctagattg gacagagaaa
 1261 ctcatactga taagtgagg attgtcagga aataaggcag gcatctagcc tcgcattaag

```

1321 atgaggtata gaaggcaact gatacatact aagtgtctcaa aaaatattaa ctccctgtcc
1381 tccatcatgg ctcaagaaaa tacaacagct gagcacaccc acgggttgct tactatttac
1441 ttatcagttt agtgtatctt attttgtttc catgtgaatt tacttgtgaa gagatgactg
1501 gattctctcc agagatagga agatccctcc tggtttaatt cctaccttta tttatttatt
1561 tttcaattag actcaggtat tgataaaaat tcaaagtca gattacaaag gtgtgtggga
1621 tttttcttcc cacgttacac aatttaagtc gactgttttc agatcaaaac tcaagacaac
1681 tccttcacca catttcctgt ttgtaactga aacaaagtac acacaaaaga ttttaagaaa
1741 cagaagagaa aagaatccga ggacacagata aagataagtt ttactgtcat gctgctttta
1801 acataacaga gcaacatcac ctaggaaaaa agtttgtagg aggattttta atccatatat
1861 ttgtcttatg gctagataaa gatttctccg aaaaaaagaa gcatg
//

```

Now parse GenBank

To Parse the genbank file - it is the same code! Just change the format.

```

import sys
import Bio
from Bio import SeqIO
from Bio.Seq import Seq

# seqfile
filename = "/bigdata/gen220/shared/data/AJ240084.gbk"
for seq_record in SeqIO.parse( filename , "genbank"):
    print(seq_record.id)
    print(repr(seq_record.seq))
    print(seq_record.seq)
    print(len(seq_record))

```

Produces

```

AJ240084.1
Seq('CCAAAAATTTCCAGTCCTGAAACCCCTTTCTCTTTCCAATGTCCTCTGTAAGCTC...ATG')
CCAAAAATTTCCAGTCCTGAAACCCCTTTCTCTTTCCAATGTCCTCTGTAAGCTCGAGTTGTGGGCATCTACTTTGCCCATATTCCAAGGTC
1905

```

Parse features from GenBank file

See documentation on [SeqIO here](#) and the [tutorial](#)

```

#!/usr/bin/env python3
import sys
import Bio
from Bio import SeqIO
from Bio.Seq import Seq
filename = sys.argv[1]

```

```

for seq_record in SeqIO.parse( filename , "genbank"):
    print(seq_record.id)
    for feature in seq_record.features:
        print("\t",feature.type,feature.location)
        print("\t",feature.type,feature.location.start,
              feature.location.end, feature.location.strand)

```

An even simpler fasta parser to dictionary

Here's a FASTA data file you have downloaded before - vector sequences in NCBI BLAST database.

```

$ head /bigdata/gen220/shared/data/vector
>gi|413493|emb|A08585.1| pMG36 expression vector DNA sequence
ATGGCAATCGTTTCAGCAGAAAAATTCGTAATTCGAGCTCGCCCGGGGATCGATCCTCTAGAGTCGACCT
GCAGGCATGCAAGCTT
>gi|513652|emb|A13775.1| oligonucleotide 14
CGAATCGTAACCGTTCGTACGAGAATCGCTGTCCTCTCCTTC

from Bio import SeqIO
infile="/bigdata/gen220/shared/data/vector"
handle = open(infile, "r")
record_dict = SeqIO.to_dict(SeqIO.parse(handle, "fasta"))
print(list(record_dict.keys())[:2]) # print the first two record IDs
handle.close()
print(record_dict["gi|513652|emb|A13775.1|"]) #use any record ID (no '>')

['gi|413493|emb|A08585.1|', 'gi|513652|emb|A13775.1|']
ID: gi|513652|emb|A13775.1|
Name: gi|513652|emb|A13775.1|
Description: gi|513652|emb|A13775.1| oligonucleotide 14
Number of features: 0
Seq('CGAATCGTAACCGTTCGTACGAGAATCGCTGTCCTCTCCTTC')

```

Convert file formats

From GenBank to Fasta

```

from Bio import SeqIO
infile="/bigdata/gen220/shared/data/cor6_6.gbk"
input_handle = open(infile, "rt")
output_handle = open("cor6_6.fasta", "w")

sequences = SeqIO.parse(input_handle, "genbank")
count = SeqIO.write(sequences, output_handle, "fasta")

output_handle.close()
input_handle.close()

```

Or even more simply

```
from Bio import SeqIO
infile="/bigdata/gen220/shared/data/cor6_6.gbk"
count = SeqIO.convert(infile, "genbank", "cor6_6.fasta", "fasta")
print("Converted %i records" % count)
```

Process URL retrieved data to Bio.SeqIO

```
#!/usr/bin/env python3
from urllib.request import urlopen
from io import TextIOWrapper
from Bio import SeqIO
url="https://www.uniprot.org/uniprot/P10127.fasta"
handle = TextIOWrapper(urlopen(url))

sequences = SeqIO.parse(handle, "fasta")
for seq in sequences:
    print("seq is ",seq.id, " ", seq.seq)
```

Other BioPython modules

Pairwise Alignment Parsing (BLAST, FASTA, HMMER)

Multiple Alignment Parsing

Database access (local, fast indexed files; Remote databases via Web)

Some Graphics drawing support

GFF parsing - http://biopython.org/wiki/GFF_Parsing

Remember to have installed the bcbio-gff package in your local conda environment.

```
#!/usr/bin/env python3
import os
gffurl="https://fungidb.org/common/downloads/release-45/ScerevisiaeS288c/gff/data/FungiDB-45"
in_file = os.path.basename(gffurl)

if not os.path.exists(in_file):
    os.system("curl -O "+gffurl)

from BCBio import GFF
filter_options = dict(
    gff_id = ["BK006934"], # this specifies the names of the chromosome/contigs you want
    gff_type = ["CDS"], # this limits the type of features to include
    gff_source = ["EuPathDB"])
```

```

# this option uses the limit_info value
# filter_options is a dictionary that specifies the subset of features to print out
in_handle = open(in_file)
for rec in GFF.parse(in_handle, limit_info=filter_options):
    for feat in rec.features:
        # this prints out the feature object
        # this prints out a default options which provides some default printing
        print(feat)
        break # this stops after the first feature, just printing out
in_handle.close()

in_handle = open(in_file)
# this is same as above but doesn't use the filter
for rec in GFF.parse(in_handle):
    for feat in rec.features:
        print(feat)
        break # this stops after the first feature, just printing out
in_handle.close()

```